Sesión 17: Arquitectura con Riverpod: Fundamentos (ProviderScope, Tipos de Providers: Provider, StateProvider, NotifierProvider)

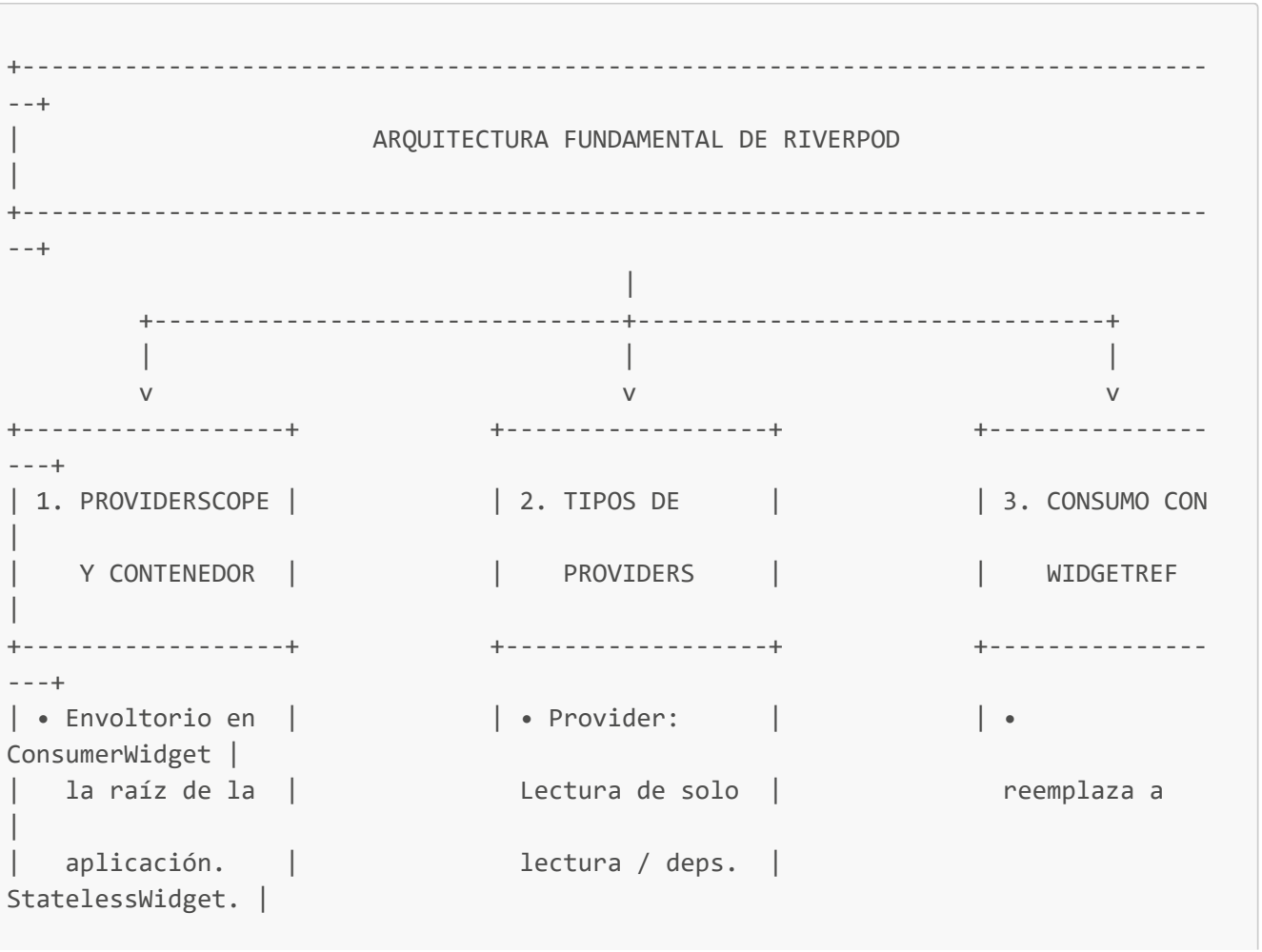
Introducción

En las sesiones anteriores comprendimos la necesidad de abstraer la gestión de estado más allá del entorno nativo de `setState()` e `InheritedWidget`. Riverpod es una de las soluciones modernas y sólidas dentro del ecosistema de Flutter para la gestión de estado y la inyección de dependencias. Diseñado por Remi Rousselet (creador de `provider`), Riverpod resuelve limitaciones históricas al eliminar la dependencia estricta del `BuildContext`, garantizar seguridad en tiempo de compilación y permitir la lectura o combinación de estados de forma declarativa.

En esta sesión abordaremos la arquitectura fundamental de Riverpod: el contenedor global `ProviderScope`, el consumo de estado mediante `ConsumerWidget` / `WidgetRef` y el uso de los tres proveedores esenciales para datos síncronos y reactivos: `Provider`, `StateProvider` y `NotifierProvider`.

Conceptos y Definiciones

Riverpod desacopla el estado del árbol de widgets, almacenándolo en contenedores independientes declarados de forma global (top-level) pero administrados de forma scoped y segura.



```
| • Almacena el    |            | • StateProvider: |            | • ref.watch():
|
|   estado real de |              Estado simple    |              Suscripción.
|
|   los proveedores|              y mutable.       |            | • ref.read():
|
| • Permite overrides|          | • NotifierProvider|              Lectura
puntual. |
|   en pruebas.    |              Lógica compleja  |            | •
ref.listen():  |
|                  |              e inmutable.     |              Efectos
primarios|
+------------------+            +------------------+            +--------------
---+
```

1. `ProviderScope`: El Contenedor Global

- `ProviderScope` es un widget que debe envolver la raíz de la aplicación (encima de `MaterialApp`).
- Se encarga de almacenar el estado real de cada `Provider` instanciado en la aplicación.
- Permite sobreescribir el comportamiento de los proveedores en pruebas unitarias o de integración (*overrides*).

---

2. Tipos de Providers Fundamentales

- `Provider` **(Solo lectura / Valores calculados):**
- Ideal para acceder a servicios inmutables, repositorios, configuraciones o valores derivados.
- *Ejemplo:* Inyección de una clase de servicio de API o cálculo de un total con impuestos.
- `StateProvider` **(Estado simple mutable):**
- Diseñado para almacenar valores primitivos simples (booleanos, enteros, cadenas) donde la lógica de modificación no requiere métodos complejos.
- *Ejemplo:* Conmutador de modo oscuro (`true`/`false`), un filtro de búsqueda o el índice de un filtro seleccionado.
- `NotifierProvider` **(Lógica de negocio encapsulada con** `Notifier`**):**
- Es la herramienta estándar para gestionar estados complejos o estructurados en versiones modernas de Riverpod (Riverpod 2.x/3.x).
- Separa la lógica de negocio (clase que extiende de `Notifier<T>`) del estado que expone (tipo `T` inmutable).
- Encapsula métodos de mutación claros, promoviendo la inmutabilidad y la separación de responsabilidades.

---

3. Consumo de Estado: `ConsumerWidget` y `WidgetRef`

Para interactuar con un proveedor desde la interfaz de usuario:

- Se reemplaza `StatelessWidget` por `ConsumerWidget`, el cual proporciona el objeto `WidgetRef ref` en su método `build`.
- **Métodos principales de `WidgetRef`:**
- `ref.watch(provider)`: Escucha el proveedor y re-ejecuta el método `build()` cuando el estado cambia (suscripción reactiva).
- `ref.read(provider)`: Lee el estado actual o accede a la clase del proveedor sin suscribir al widget (ideal para eventos de botones u `onPressed`).
- `ref.listen(provider, (prev, next) => ...)`: Reacciona a cambios de estado para ejecutar efectos secundarios (alertas, snackbars, navegación).

Ejemplo Completo con Código Fuente: Gestión de Configuración y Lista de Tareas con Riverpod

El siguiente ejemplo implementa una solución completa que integra `ProviderScope`, un `Provider` de lectura, un `StateProvider` para un filtro simple y un `NotifierProvider` para administrar una lista de tareas académicas.

> **Nota de dependencia:** Para ejecutar este ejemplo se requiere incluir `flutter_riverpod: ^2.5.1` en el archivo `pubspec.yaml`.

Código Fuente Explicado (`lib/main.dart`)

```dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

void main() {
  // 1. PROVIDERSCOPE: Requisito obligatorio en la raíz de la app
  runApp(
    const ProviderScope(
      child: DemoRiverpodApp(),
    ),
  );
}

/// Modelo Inmutable para Tareas Académicas
@immutable
class TareaAcademica {
  final String id;
  final String titulo;
  final bool completada;

  const TareaAcademica({
    required this.id,
    required this.titulo,
    this.completada = false,
  });
```

```
  TareaAcademica copyWith({String? id, String? titulo, bool? completada}) {
    return TareaAcademica(
      id: id ?? this.id,
      titulo: titulo ?? this.titulo,
      completada: completada ?? this.completada,
    );
  }
}

// ============================================================================
// DEFINICIÓN DE PROVEEDORES (GLOBALES Y DECLARATIVOS)
// ============================================================================

/// 1. PROVIDER (Lectura Inmutable): Inyección del título de la materia
final tituloMateriaProvider = Provider<String>((ref) {
  return 'Desarrollo de Aplicaciones Móviles - ESCOM';
});

/// 2. STATEPROVIDER (Estado Simple): Filtro de visualización (Mostrar solo
pendientes)
final soloPendientesProvider = StateProvider<bool>((ref) => false);

/// 3. NOTIFIER (Lógica de Negocio): Administrador de la lista de tareas
class TareasNotifier extends Notifier<List<TareaAcademica>> {
  @override
  List<TareaAcademica> build() {
    // Estado inicial de la lista
    return const [
      TareaAcademica(id: '1', titulo: 'Completar Práctica de CustomPainter'),
      TareaAcademica(id: '2', titulo: 'Estudiar patrón InheritedWidget'),
      TareaAcademica(id: '3', titulo: 'Implementar ProviderScope en Riverpod',
completada: true),
    ];
  }

  void agregarTarea(String titulo) {
    if (titulo.trim().isEmpty) return;
    final nuevaTarea = TareaAcademica(
      id: DateTime.now().toString(),
      titulo: titulo.trim(),
    );
    // Asignación inmutable produciendo una nueva lista
    state = [...state, nuevaTarea];
  }

  void alternarEstadoTarea(String id) {
    state = [
      for (final tarea in state)
        if (tarea.id == id)
          tarea.copyWith(completada: !tarea.completada)
        else
          tarea,
    ];
```

```
  }
}

/// 3. NOTIFIERPROVIDER: Expone el TareasNotifier a la interfaz
final tareasProvider = NotifierProvider<TareasNotifier, List<TareaAcademica>>(
  TareasNotifier.new,
);

/// Provider derivado (Calculado): Filtra la lista en función del StateProvider
final tareasFiltradasProvider = Provider<List<TareaAcademica>>((ref) {
  final tareas = ref.watch(tareasProvider);
  final soloPendientes = ref.watch(soloPendientesProvider);

  if (soloPendientes) {
    return tareas.where((t) => !t.completada).toList();
  }
  return tareas;
});

// ============================================================================
// INTERFAZ DE USUARIO (CONSUMERWIDGETS)
// ============================================================================

class DemoRiverpodApp extends StatelessWidget {
  const DemoRiverpodApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'ESCOM - Fundamentos de Riverpod',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        useMaterial3: true,
        colorScheme: ColorScheme.fromSeed(
          seedColor: const Color(0xFF6B112D), // Guinda Institucional
        ),
      ),
      home: const PantallaTareasRiverpod(),
    );
  }
}

/// Pantalla Principal reemplaza StatelessWidget por ConsumerWidget
class PantallaTareasRiverpod extends ConsumerWidget {
  const PantallaTareasRiverpod({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    // Lectura de Provider inmutable
    final tituloMateria = ref.watch(tituloMateriaProvider);
    // Lectura de lista calculada filtrada
    final tareas = ref.watch(tareasFiltradasProvider);
    // Lectura del valor del StateProvider
    final soloPendientes = ref.watch(soloPendientesProvider);
```

```
    return Scaffold(
      appBar: AppBar(
        title: const Text('Gestión de Tareas Riverpod'),
        backgroundColor: Theme.of(context).colorScheme.primary,
        foregroundColor: Colors.white,
      ),
      body: Padding(
        padding: const EdgeInsets.all(16.0),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(
              tituloMateria,
              style: const TextStyle(fontSize: 16, fontWeight: FontWeight.bold),
            ),
            const SizedBox(height: 8),

            // Controlde Filtro usando StateProvider
            Row(
              children: [
                const Text('Mostrar solo pendientes:'),
                const Spacer(),
                Switch(
                  value: soloPendientes,
                  onChanged: (valor) {
                    // Actualización directa del StateProvider mediante .notifier
                    ref.read(soloPendientesProvider.notifier).state = valor;
                  },
                ),
              ],
            ),
            const Divider(),

            // Lista de Tareas
            Expanded(
              child: tareas.isEmpty
                  ? const Center(child: Text('No hay tareas para mostrar.'))
                  : ListView.builder(
                      itemCount: tareas.length,
                      itemBuilder: (context, index) {
                        final tarea = tareas[index];
                        return CheckboxListTile(
                          title: Text(
                            tarea.titulo,
                            style: TextStyle(
                              decoration: tarea.completada
                                  ? TextDecoration.lineThrough
                                  : null,
                            ),
                          ),
                          value: tarea.completada,
                          onChanged: (_) {
                            // Invocación del método de negocio mediante .notifier
```

```
                                ref
                                    .read(tareasProvider.notifier)
                                    .alternarEstadoTarea(tarea.id);
                              },
                            );
                          },
                        ),
                    ),
                  ],
                ),
              ),
              floatingActionButton: FloatingActionButton(
                onPressed: () => _mostrarDialogoNuevaTarea(context, ref),
                child: const Icon(Icons.add),
              ),
            );
          }

          void _mostrarDialogoNuevaTarea(BuildContext context, WidgetRef ref) {
            final controller = TextEditingController();
            showDialog(
              context: context,
              builder: (context) => AlertDialog(
                title: const Text('Nueva Tarea Académica'),
                content: TextField(
                  controller: controller,
                  decoration: const InputDecoration(
                    hintText: 'Ej. Entregar reporte de laboratorio',
                  ),
                  autofocus: true,
                ),
                actions: [
                  TextButton(
                    onPressed: () => Navigator.pop(context),
                    child: const Text('Cancelar'),
                  ),
                  ElevatedButton(
                    onPressed: () {
                      // Uso de ref.read para ejecutar la mutación de estado
                      ref.read(tareasProvider.notifier).agregarTarea(controller.text);
                      Navigator.pop(context);
                    },
                    child: const Text('Agregar'),
                  ),
                ],
              ),
            );
          }
        }
```

Prácticas Guiadas / Laboratorio

Práctica 1: Creación de un Contador de Caracteres con `StateProvider`

1. Declara un `StateProvider<String>` encargado de almacenar el texto de un campo de búsqueda en tiempo real.
2. Agrega un `TextField` que actualice dicho `StateProvider` en su evento `onChanged` mediante `ref.read(busquedaProvider.notifier).state = val`.
3. Crea un `Provider` secundario derivado que calcule la longitud de la cadena y muestre una etiqueta con la cantidad de caracteres en la pantalla.

Práctica 2: Gestión de Carrito de Compras con `NotifierProvider`

1. Define una clase `CarritoNotifier` que extienda de `Notifier<List<Producto>>`.
2. Implementa métodos en el `Notifier` para agregar productos, eliminar por ID y limpiar la lista de forma inmutable.
3. Conecta la vista con un `ConsumerWidget` y valida con el depurador que `ref.watch(carritoProvider)` reconstruye la vista únicamente al modificar los elementos del carrito.

Resumen y Puntos Clave

- **Desacoplamiento de `BuildContext`:** Riverpod permite declarar proveedores de estado de forma global sin depender directamente de la jerarquía de widgets.
- **`ProviderScope`:** Envoltorio obligatorio en la raíz que almacena las instancias de los proveedores y administra su ciclo de vida.
- **`Provider` vs `StateProvider` vs `NotifierProvider`:**
- `Provider`: Datos inmutables o servicios de lectura.
- `StateProvider`: Mutación de variables primitivas simples.
- `NotifierProvider`: Lógica de negocio encapsulada con estado inmutable estructurado.
- **Uso de `WidgetRef`:** `ref.watch` para reaccionar a cambios en `build()`, `ref.read` para eventos puntuales de interacción.

Bibliografía y Referencias

1. **Rousselet, R.** (2024). *Riverpod Documentation - Reactive State-Management Framework*. Recuperado de: https://riverpod.dev
2. **Google LLC.** (2024). *State Management Options in Flutter*. Recuperado de: https://docs.flutter.dev/data-and-backend/state-mgmt/options#riverpod
3. **Moedano, B.** (2023). *Desarrollo de Aplicaciones Móviles Adaptativas con Flutter*. Editorial Alfaomega. ISBN: 978-6075389102.